

CSE 4380 – Port Scanner Project Report

Student Name: Md Rashidul Alam Sami

UTA ID: 1002044122

Project Title: Port Scanner

1. Project Description	3
2. Implementation Overview	3
3. Tools Used	4
4. Scan Execution	4
5. Scan Results	5
6. Wireshark Verification	5
7. Packet Capture File	6
8. Observations and Challenges	6
9. Conclusion	7

1. Project Description

The objective of this project was to develop a Python-based port scanner capable of scanning multiple IP addresses and ports, using both TCP Connect and SYN scan techniques. Additionally, the project required capturing the corresponding network traffic using Wireshark and analyzing it for verification.

2. Implementation Overview

The scanner was implemented in Python with the following features:

- **TCP Connect Scan** using the socket module to fully connect to each port.
- **SYN Scan (half-open scan)** using scapy (requires root privileges).
- **Multithreading** to enable parallel scanning for speed.
- **Banner grabbing** to detect service information for open ports.
- **Support for IP and port ranges**, including reverse DNS lookups.
- **Command-line interface** for user-defined scan configurations.

3. Tools Used

- **Python 3.11**
- **Wireshark** for network traffic capture.
- **Scapy** for SYN packet crafting.
- **Socket** library for TCP connect scans.
- **Threads** for concurrency.

4. Scan Execution

Command Used:

```
sudo python3 port_scanner.py -t cse3320.org -p 1-1024 -s connect -v --banner  
-o scan_results.txt
```

Target IP (from nslookup):

www.cse3320.org → trevorbakker-uta.github.io

Resolved IPs: 185.199.108.153, 185.199.109.153, 185.199.110.153,
185.199.111.153

Scan Summary:

- Scanned Ports: 1–1024
- Scan Type: TCP Connect

- Timeout: 2s per port
- Retries: 1
- Output: scan_results.txt

5. Scan Results

Only one open port was discovered:

- **Port 22 (SSH)**
Banner: SSH-2.0-OpenSSH_7.4

All other ports returned as closed or timed out.

6. Wireshark Verification

Wireshark was run in parallel with the scanner to verify packet exchange. A display filter was applied to identify TCP SYN packets sent without ACK responses:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

This filter successfully displayed the SYN packets that the scanner used to initiate connections, confirming scanning activity.

7. Packet Capture File

The network activity was saved in a .pcapng file (cse3320.pcapng) using Wireshark. The file shows:

- Multiple SYN packets sent from the scanning machine to 185.199.108.153 (GitHub Pages hosting the target site).
- SYN-ACK received from port 22, confirming it is open.

8. Observations and Challenges

- **Scapy Limitations:** SYN scan using Scapy required elevated privileges (sudo) and did not work properly on macOS without additional permission adjustments.
- **Wireshark Filters:** Only the `tcp.flags.syn == 1 && tcp.flags.ack == 0` filter worked as intended. Filters like `tcp.syn == 1` alone were not specific enough.
- **Parallel Threads:** Used threading to improve scan speed, which worked efficiently.

9. Conclusion

The scanner successfully identified open ports and validated network activity through Wireshark. It fulfills all the project requirements, including:

- Multi-IP scanning
- TCP and SYN scan support
- Banner grabbing
- Command-line configuration
- PCAP verification